

Supervised learning: linear models

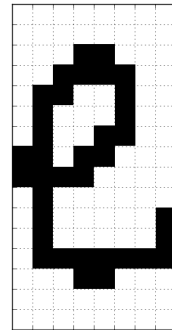
Plan

- Algorithme k plus proche voisin
 - Algorithme
 - Exemple
- Classification linéaire avec le perceptron
 - Rappel : problème de classification
 - Algorithme de Perceptron
 - Exemple
 - Propriétés du perceptron
- Minimisation de perte
 - Apprentissage comme la minimisation de la perte
 - Algorithme de descente de gradient
 - Convexité / Analyse de descente de gradient / descente de gradient stochastique
 - Régression logistique
 - Classification

K plus proches voisins

Représentation des données

- L'**entrée (x)** est représentée par un vecteur de valeurs d'attributs réels (représentation factorisée)
 - ex.: une image est représentée par un vecteur contenant la valeur de chacun des pixels



```
array([ 0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  
        0.,  0.,  0.,  0.,  0.,  0.,  1.,  1.,  0.,  0.,  0.,  0.,  0.,  
        1.,  1.,  1.,  1.,  0.,  0.,  0.,  1.,  1.,  0.,  0.,  1.,  0.,  
        0.,  0.,  1.,  0.,  0.,  0.,  1.,  0.,  0.,  0.,  1.,  0.,  0.,  
        1.,  1.,  0.,  0.,  1.,  1.,  0.,  1.,  1.,  0.,  0.,  0.,  1.,  
        1.,  1.,  1.,  0.,  0.,  0.,  0.,  0.,  1.,  0.,  0.,  0.,  0.,  
        0.,  0.,  0.,  1.,  0.,  0.,  0.,  0.,  0.,  1.,  0.,  1.,  0.,  
        0.,  0.,  0.,  0.,  1.,  0.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  
        0.,  0.,  0.,  1.,  1.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  
        0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.]
```

- La **sortie désirée** ou **cible (y)** aura une représentation différente selon le problème à résoudre:
 - problème de classification en (C) classes: valeur discrète (index de 0 à $C-1$)
 - problème de régression: valeur réelle ou continue

Illustration: 3 plus proches voisins

- Reconnaissance de caractère: est-ce un 'e' ou un 'o'?

Ensemble d'entraînement

(100 exemples d'apprentissage par classe)

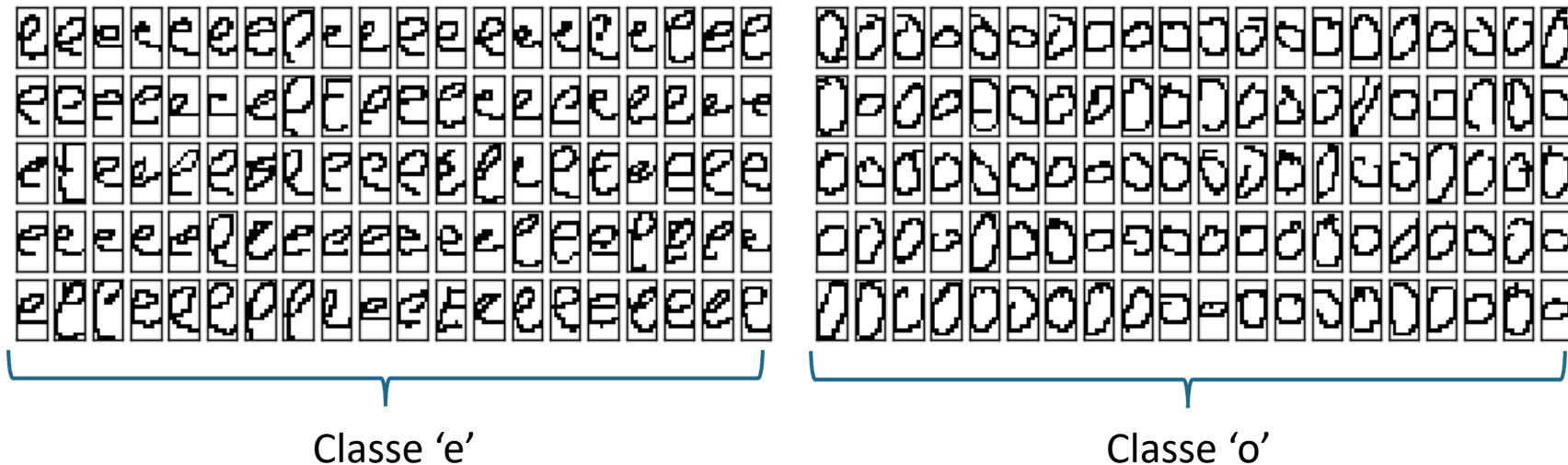
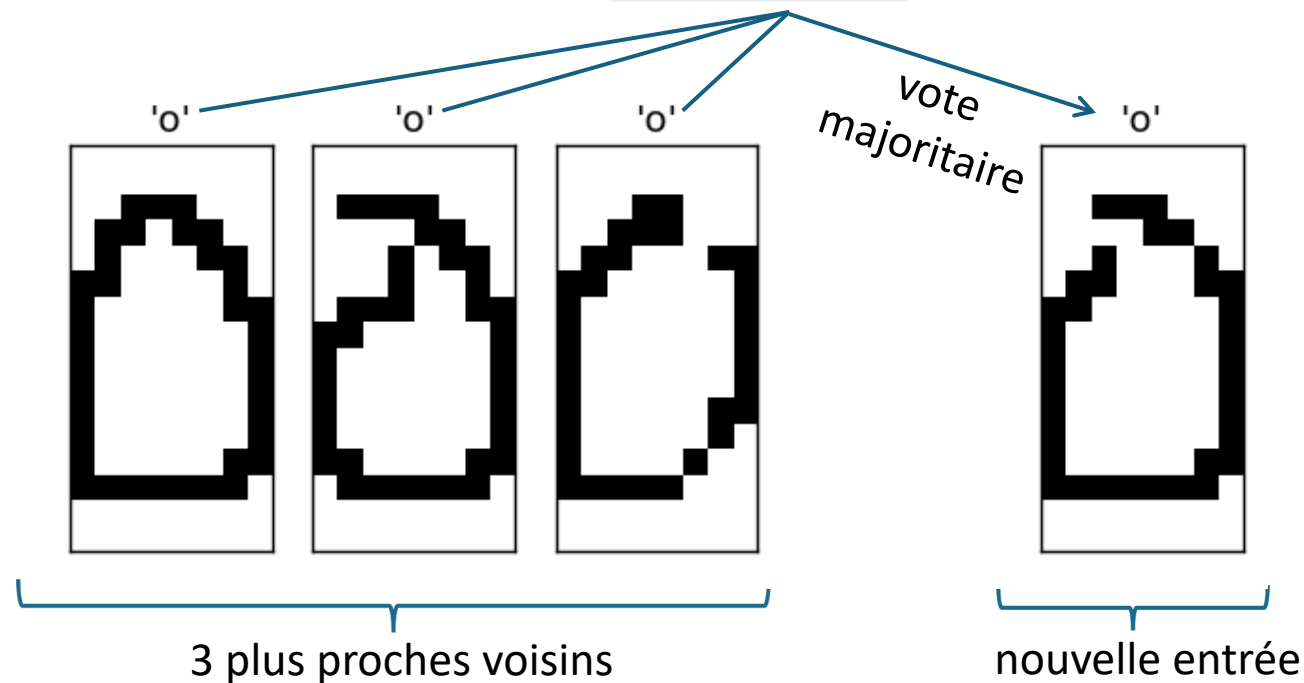


Illustration: 3 plus proches voisins

- Reconnaissance de caractère: est-ce un 'e' ou un 'o'?



Vrai!

Illustration: 3 plus proches voisins

- Reconnaissance de caractère: est-ce un 'e' ou un 'o'?

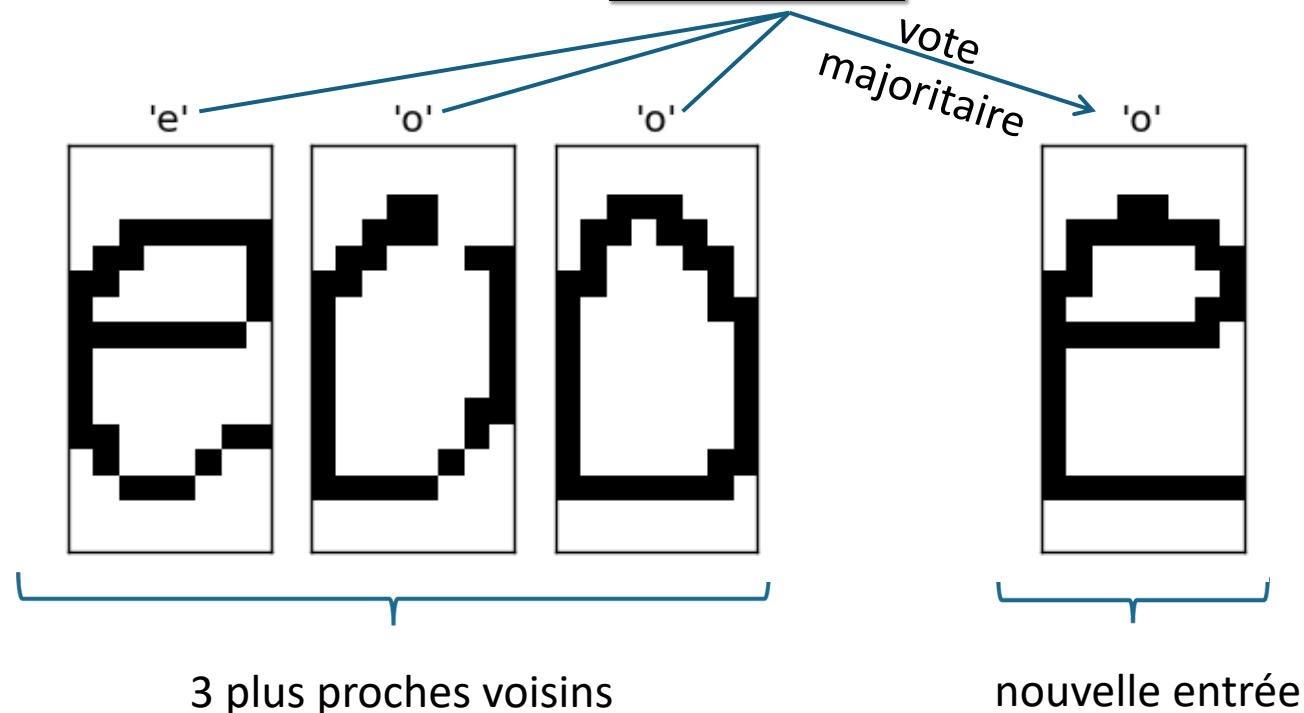
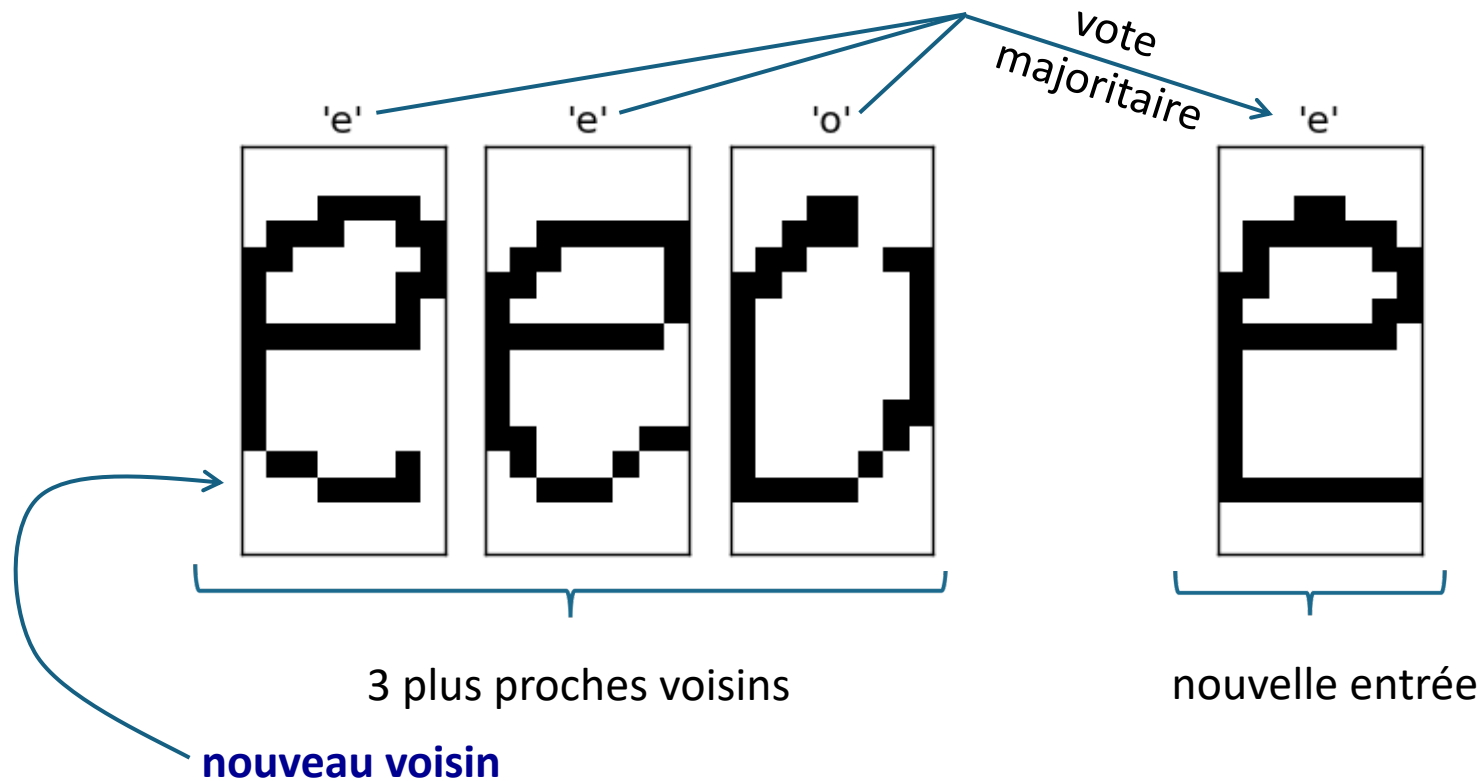


Illustration: 3 plus proches voisins

- Reconnaissance de caractère: est-ce un 'e' ou un 'o'?



Si on ajoute
200 exemples
par classe...

Vrai!

Exemple: classifieur k plus proches voisins

- Possiblement l'algorithme d'apprentissage de classification le plus simple
- **Idée:** étant donnée une entrée ($\mathbf{X}$)
 1. trouver les k entrées ($\mathbf{x}_t$) parmi les exemples d'apprentissage qui sont les plus « proches » de ($\mathbf{X}$)
 2. faire voter chacune de ces entrées pour leur classe associée ($\mathbf{y}_t$)
 3. retourner la classe majoritaire
- Le succès de cet algorithme va dépendre de deux facteurs
 - la quantité de données d'entraînement (plus il y en a, meilleure sera la performance)
 - la qualité de la mesure de distance (est-ce que deux entrées jugées similaires sont de la même classe?)
 - en pratique, on utilise souvent la distance Euclidienne:

$$d(\mathbf{x}_1, \mathbf{x}_2) = \sqrt{\sum_k (x_{1,k} - x_{2,k})^2}$$

$\mathbf{X}$ = vecteur
 x = scalaire

Rappel - Apprentissage supervisé

- Un problème d'apprentissage supervisé est formulé comme suit:

- « Étant donné un **ensemble d'entraînement** de N exemples:

$$(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N) \} D$$

- où chaque $(\mathbf{y}_j)$ a été généré par une **fonction inconnue** $[\mathbf{y}=\mathbf{f}(\mathbf{x})]$, découvrir une nouvelle fonction **(h)** (**modèle** ou **hypothèse**) qui sera une bonne approximation de **(f)** (c'est à dire $[\mathbf{h}(\mathbf{x})=\mathbf{f}(\mathbf{x})]$)
- Un algorithme d'apprentissage peut donc être vu comme étant une fonction **(A)** à laquelle on donne un ensemble d'entraînement et qui donne en retour cette fonction : **A(D)=h**

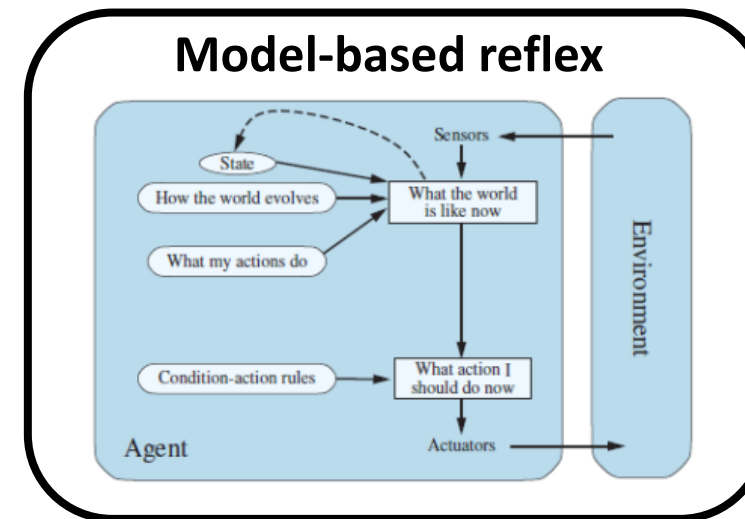
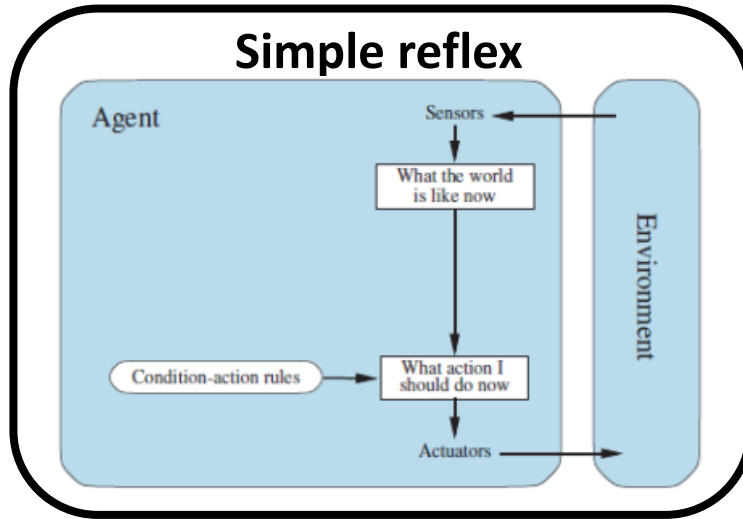
Retour sur classifieur k plus proches voisins

- Dans le cas de l'algorithme k plus proches voisins:
 - $(\mathbf{A})$ est un programme qui produit lui-même un programme, soit celui qui fait une prédiction à l'aide de la procédure k plus proches voisins
 - $[\mathbf{h}=\mathbf{A}(\mathbf{D})]$ est le programme qui fait voter les k plus proches voisins dans $(\mathbf{D})$ d'une entrée donnée
 - $\mathbf{h}(\mathbf{x})$ est la sortie du programme pour l'entrée $(\mathbf{x})$, c'est à dire une prédiction de la classe de $\mathbf{x}$
 - $(\mathbf{f})$ est la « fonction » qui a généré nos données d'entraînement
 - ex.: l'être humain qui a étiqueté les images de caractères
- On peut démontrer que plus $(\mathbf{D})$ est grand, plus $(\mathbf{h})$ sera une bonne approximation de $(\mathbf{f})$
 - en augmentant la taille de l'ensemble d'entraînement, les k plus proches voisins ne peuvent changer qu'en étant encore plus proches (plus similaires) à l'entrée

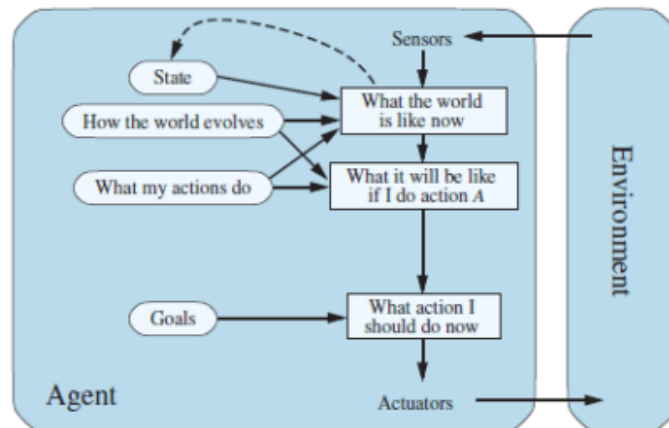
Mesure de la performance d'un algorithme d'apprentissage

- Comment évaluer le succès d'un algorithme?
 - on pourrait regarder l'erreur moyenne commise sur les exemples d'entraînement, mais cette erreur sera nécessairement optimiste
 - (**h**) a déjà vu la bonne réponse pour ces exemples!
 - on mesure donc seulement la capacité de l'algorithme à **mémoriser**
- Ce qui nous intéresse vraiment, c'est la capacité de l'algorithme à **généraliser** sur de **nouveaux exemples**
 - ça reflète mieux le contexte dans lequel on va utiliser (**h**)
- Pour mesurer la généralisation, on met de côté des exemples étiquetés, qui seront utilisés seulement à la toute fin, pour calculer l'erreur
 - on l'appelle l'**ensemble de test**

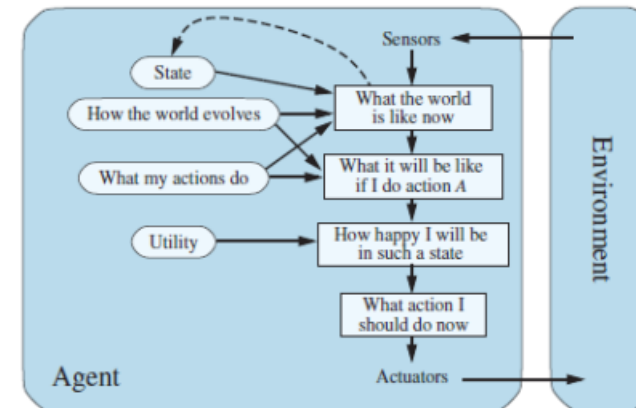
Algorithme des K plus proches voisins pour quel type d'agent?



Goal-based



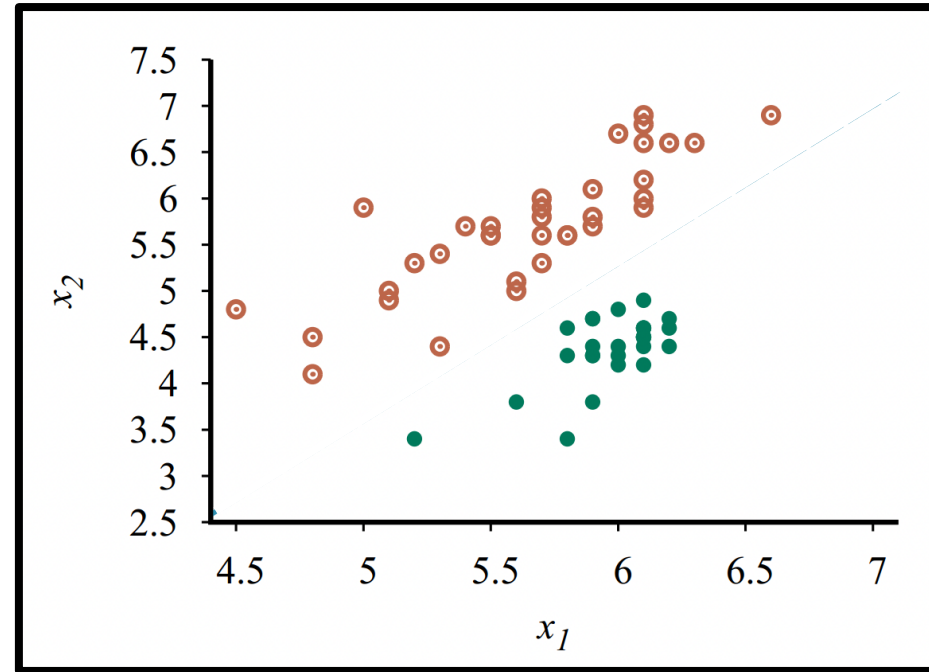
Utility-based



Classification linéaire avec le perceptron

Example

Sample x_i	x_{i1}	x_{i2}	y_i
x_1	4.5	4.7	0
x_2	4.7	4.1	0
x_3	4.7	4.9	0
x_4	5	5.9	0
x_5	5.1	4.9	0
x_6	5.1	5	0
x_7	5.2	3.5	1
..	..	..	..

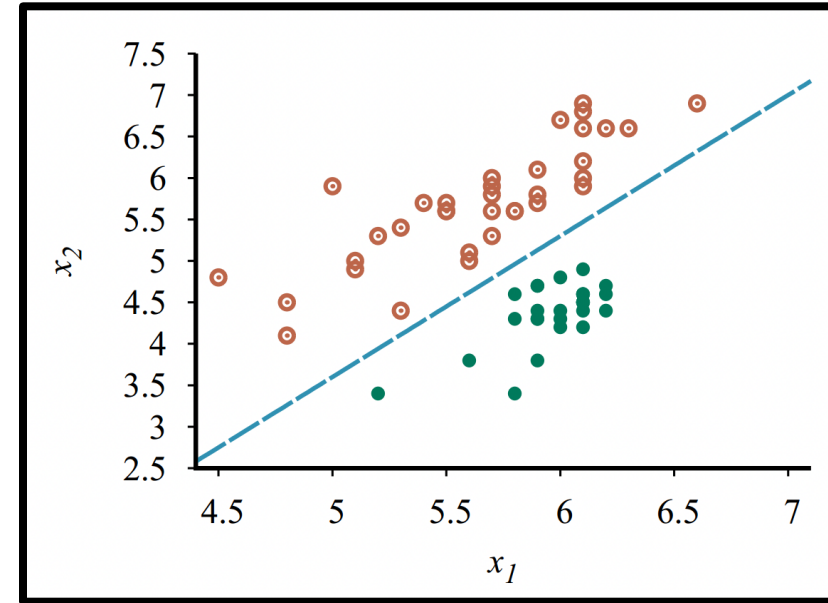


Exemple

Sample x_i	x_{i1}	x_{i2}	y_i
x_1	4.5	4.7	1
x_2	4.7	4.1	1
x_3	4.7	4.9	1
x_4	5	5.9	1
x_5	5.1	4.9	1
x_6	5.1	5	1
x_7	5.2	3.5	0
..	..	..	..

$$\mathbf{w} = [w_0, w_1, w_2]$$

$$\mathbf{x} = [1, x_1, x_2]$$



Équation de la droite, à peu près: $-2.6 - 0.2x_1 + x_2 = 0$

En général: $w_0 + w_1 x_1 + w_2 x_2 = 0$

Si $w_0 + w_1 x_1 + w_2 x_2 \geq 0$ alors $y = 1$ sinon $y = 0$

$\mathbf{w} \cdot \mathbf{x}$

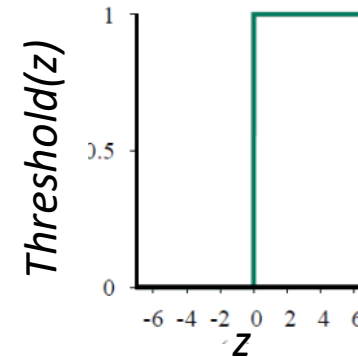
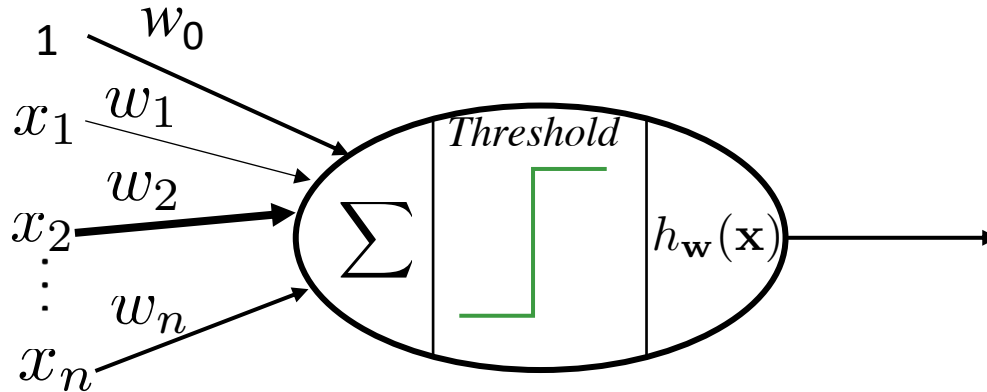
Threshold($\mathbf{w} \cdot \mathbf{x}$)

Algorithme Perceptron(Rosenblatt, 1957)

- Un des plus vieux algorithmes de classification
- **Idée**: modéliser la décision à l'aide d'une fonction linéaire, suivi d'un seuil:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x}) = \text{Threshold}(\sum_i w_i x_i)$$

où $\text{Threshold}(z) = 1 \quad \text{si } z \geq 0 \quad ; \quad \text{SINON } \text{Threshold}(z) = 0$



- Le **vecteur de poids** $\mathbf{w}$ correspond aux **paramètres** du modèle
- L'entrée x_0 est toujours à 1 et le poids correspondant w_0 équivaut à un biais.

Algorithme Perceptron (Rosenblatt, 1957)

- L'algorithme d'apprentissage consiste à adapter la valeur des paramètres (c'est-à-dire les poids) de façon que $h_{\mathbf{w}}(\mathbf{x})$ soit cohérent avec les données d'entraînement.

- Algorithme du Perceptron:

1. pour chaque paire $(\mathbf{x}_t, y_t) \in D$
 - a. calculer $h_{\mathbf{w}}(\mathbf{x}_t) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x}_t)$
 - b. $w_i \leftarrow w_i + \alpha(y_t - h_{\mathbf{w}}(\mathbf{x}_t))x_{t,i} \quad \forall i$ (mise à jour des poids)
2. retourner à 1 jusqu'à l'atteinte du critère d'arrêt (pour l'instant, cela veut dire le nombre maximal d'itérations atteint ou le nombre d'erreurs est 0)

- L'équation de mise à jour des poids est appelée la règle d'apprentissage du Perceptron.
- Le multiplicateur (α) est appelé le taux d'apprentissage

Algorithme Perceptron (Rosenblatt, 1957)

- Algorithme du Perceptron (forme vectorielle):
 1. pour chaque paire $(\mathbf{x}_t, y_t) \in D$
 - a. calculer $h_{\mathbf{w}}(\mathbf{x}_t) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x}_t)$
 - b. $\mathbf{w} \leftarrow \mathbf{w} + \alpha(y_t - h_{\mathbf{w}}(\mathbf{x}_t))\mathbf{x}_t$ (mise à jour des poids)
 2. retourner à 1 jusqu'à l'atteinte du critère d'arrêt (pour l'instant, cela veut dire le nombre maximal d'itérations atteint ou le nombre d'erreurs est 0)

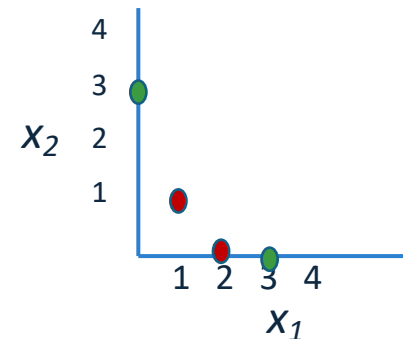
Exemple

- Simulation **avec** $\alpha = 0.1$
- Initialisation : $\mathbf{w} \leftarrow [0.5, 0, 0]$

D ensemble
entraînement

	x_{i0}	$x_i \equiv [x_{i1}, x_{i2}]$	y_i
x_1	1	[2,0]	1
x_2	1	[0,3]	0
x_3	1	[3,0]	0
x_4	1	[1,1]	1

- Paire $(\mathbf{x}_1, y_1)$:
 - $h(\mathbf{x}_1) = \text{Threshold}(\underbrace{\mathbf{w}_0 \cdot \mathbf{x}_{10} + \mathbf{w}_1 \cdot \mathbf{x}_{11} + \mathbf{w}_2 \cdot \mathbf{x}_{12}}_{\mathbf{w} \cdot \mathbf{x}_1})$
 $= \text{Threshold}(0.5) = 1$
 - puisque $h(\mathbf{x}_1) = y_1$, $\mathbf{w}$ ne change pas.



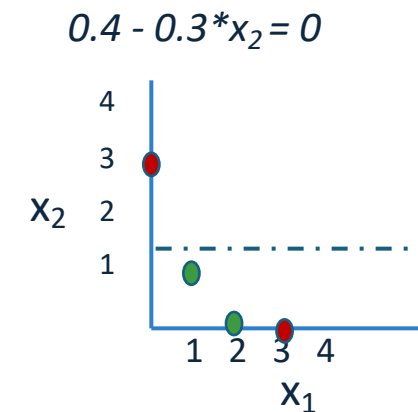
Exemple

- Simulation **avec** $\alpha = 0.1$
- Valeur courante : $\mathbf{w} \leftarrow [0.5, 0, 0]$

- Paire $(\mathbf{x}_2, y_2)$:
 - $h(\mathbf{x}_2) = \text{Threshold}(\overbrace{\mathbf{w}_0 \cdot \mathbf{x}_{20} + \mathbf{w}_1 \cdot \mathbf{x}_{21} + \mathbf{w}_2 \cdot \mathbf{x}_{22}}^{\mathbf{w} \cdot \mathbf{x}_2})$
 $= \text{Threshold}(0.5) = 1$
 - puisque $h(\mathbf{x}_2) \neq y_2$, $\mathbf{w}$ va changer:
 - $\mathbf{w} \leftarrow \mathbf{w} + \alpha (y_2 - h(\mathbf{x}_2)) \mathbf{x}_2$
 $= [0.5, 0, 0] + 0.1 * (0 - 1) [1, 0, 3]$
 $= [0.4, 0, -0.3]$

D ensemble
entraînement

	x_{i0}	$x_i \equiv [x_{i1}, x_{i2}]$	y_i
$\mathbf{x}_1$	1	[2,0]	1
$\mathbf{x}_2$	1	[0,3]	0
$\mathbf{x}_3$	1	[3,0]	0
$\mathbf{x}_4$	1	[1,1]	1



Exemple

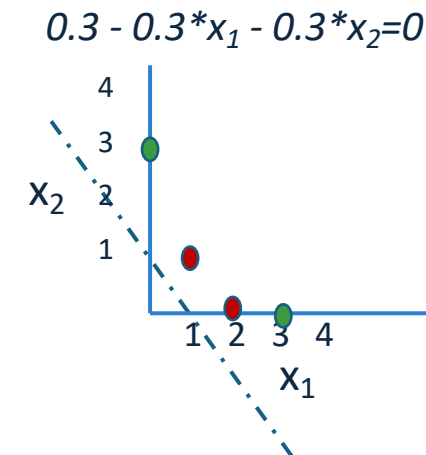
- Simulation **avec** $\alpha = 0.1$
- Valeur courante : $\mathbf{w} \leftarrow [0.4, 0, -0.3]$

- Paire $(\mathbf{x}_3, y_3)$:
 - $$h(\mathbf{x}_3) = \text{Threshold}(\overbrace{\mathbf{w}_0 \cdot \mathbf{x}_{30} + \mathbf{w}_1 \cdot \mathbf{x}_{31} + \mathbf{w}_2 \cdot \mathbf{x}_{32}}^{\mathbf{w} \cdot \mathbf{x}_3})$$

$$= \text{Threshold}(0.4) = 1$$
 - puisque $h(\mathbf{x}_3) \neq y_3$, $\mathbf{w}$ va changer:
 - $$\begin{aligned} \mathbf{w} &\leftarrow \mathbf{w} + \alpha (y_3 - h(\mathbf{x}_3)) \mathbf{x}_3 \\ &= [0.4, 0, -0.3] + 0.1 * (0 - 1) [1, 3, 0] \\ &= [0.3, -0.3, -0.3] \end{aligned}$$

D ensemble
entraînement

	x_{i0}	$x_i \equiv [x_{i1}, x_{i2}]$	y_i
$\mathbf{x}_1$	1	[2,0]	1
$\mathbf{x}_2$	1	[0,3]	0
$\mathbf{x}_3$	1	[3,0]	0
$\mathbf{x}_4$	1	[1,1]	1



Exemple

- Simulation **avec** $\alpha = 0.1$
- Valeur courante : $\mathbf{w} \leftarrow [0.3, -0.3, -0.3]$

- Paire $(\mathbf{x}_4, y_4)$:
 - $$h(\mathbf{x}_4) = \text{Threshold}(\overbrace{\mathbf{w}_0 \cdot x_{40} + \mathbf{w}_1 \cdot x_{41} + \mathbf{w}_2 \cdot x_{42}}^{\mathbf{w} \cdot \mathbf{x}_4})$$

$$= \text{Threshold}(-0.3) = 0$$
 - puisque $h(\mathbf{x}_4) \neq y_4$, $\mathbf{w}$ va changer:
 - $$\mathbf{w} \leftarrow \mathbf{w} + \alpha (y_4 - h(\mathbf{x}_4)) \mathbf{x}_4$$

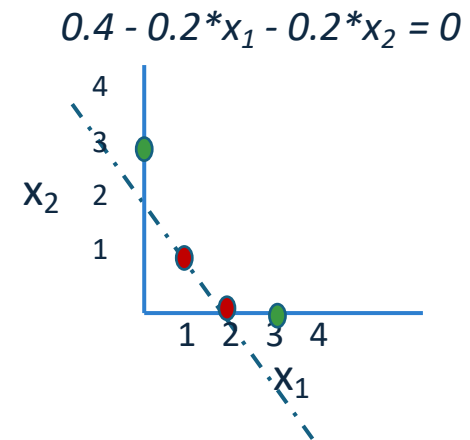
$$= [0.3, -0.3, -0.3] + 0.1 * (1 - 0) [1, 1, 1]$$

$$= [0.4, -0.2, -0.2]$$

- Et ainsi de suite, jusqu'à l'atteinte du critère d'arrêt...

ensemble
D entraînement

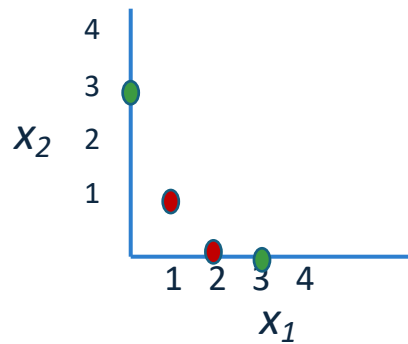
	x_{i0}	$x_i \equiv [x_{i1}, x_{i2}]$	y_i
$\mathbf{x}_1$	1	[2,0]	1
$\mathbf{x}_2$	1	[0,3]	0
$\mathbf{x}_3$	1	[3,0]	0
$\mathbf{x}_4$	1	[1,1]	1



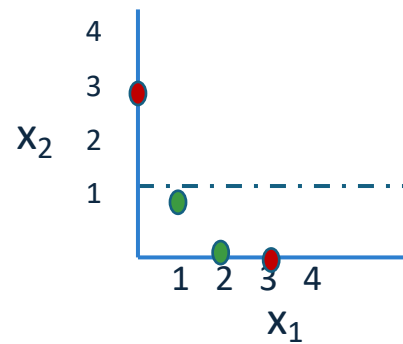
Exemple

- On voit bien que la ligne de séparation bouge avec la mise à jour des poids

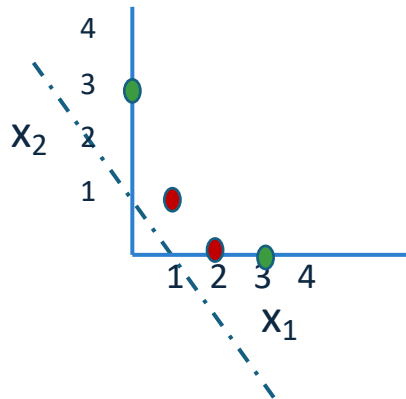
$$h(x)=0.5$$



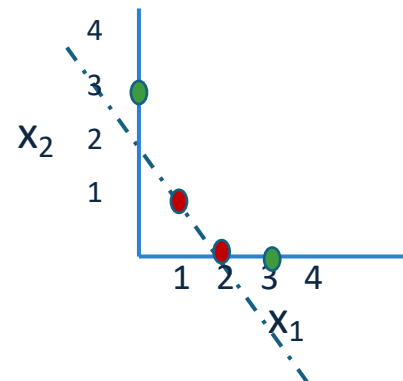
$$0.4 - 0.3 * x_2 = 0$$



$$0.3 - 0.3 * x_1 - 0.3 * x_2 = 0$$

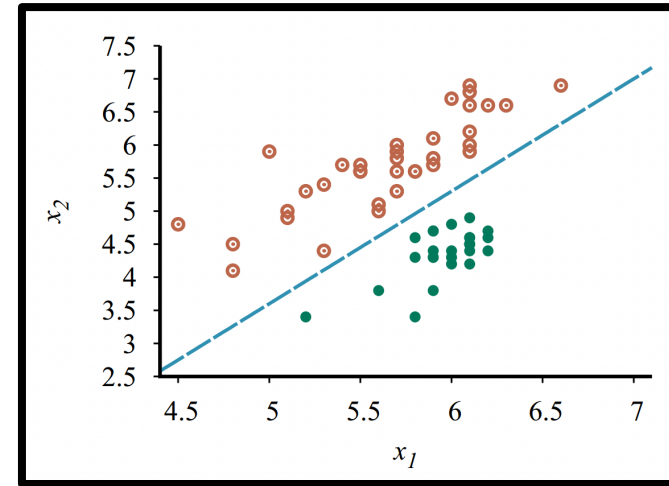


$$0.4 - 0.2 * x_1 - 0.2 * x_2 = 0$$

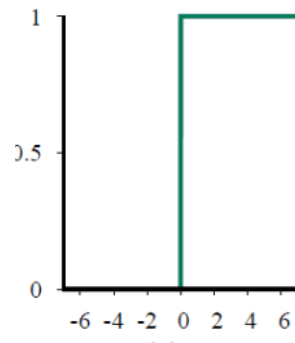


Rappel – Classification linéaire avec le perceptron

Sample x_i	x_{i1}	x_{i2}	y_i
x_1	4.5	4.7	1
x_2	4.7	4.1	1
x_3	4.7	4.9	1
x_4	5	5.9	1
x_5	5.1	4.9	1
x_6	5.1	5	1
x_7	5.2	3.5	0
..	..	..	..



$$\mathbf{w} = [w_0, w_1, w_2]$$
$$\mathbf{x} = [1, x_1, x_2]$$



Équation de la droite, à peu près: $-2.6 - 0.2x_1 + x_2 = 0$

En général: $w_0 + w_1 x_1 + w_2 x_2 = 0$

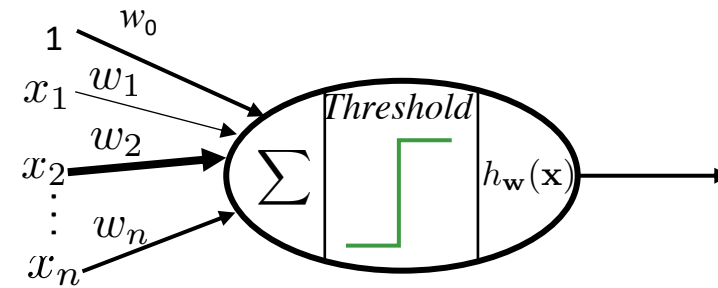
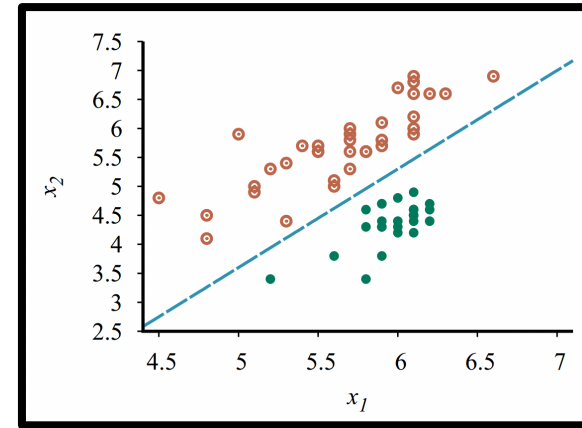
Si $w_0 + w_1 x_1 + w_2 x_2 \geq 0$ alors $y = 1$ sinon $y = 0$

$\mathbf{w} \cdot \mathbf{x}$

Threshold($\mathbf{w} \cdot \mathbf{x}$)

Rappel – Classification linéaire avec le perceptron

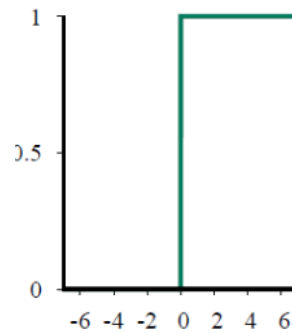
Sample x_i	x_{i1}	x_{i2}	y_i
x_1	4.5	4.7	1
x_2	4.7	4.1	1
x_3	4.7	4.9	1
x_4	5	5.9	1
x_5	5.1	4.9	1
x_6	5.1	5	1
x_7	5.2	3.5	0
..	..	..	..



$$\mathbf{w} = [w_0, w_1, w_2]$$

$$\mathbf{x} = [1, x_1, x_2]$$

$$w \approx [-2.6, 0.2, 1]$$



$$\mathbf{w} \cdot \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2$$

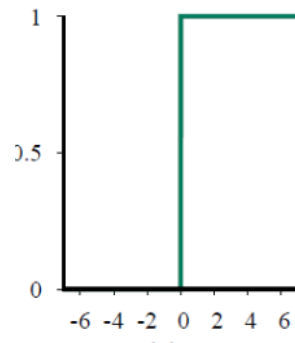
Si $\mathbf{w} \cdot \mathbf{x} \geq 0$ alors $y = 1$ sinon $y = 0$

$\text{Threshold}(\mathbf{w} \cdot \mathbf{x})$

Rappel – Classification linéaire avec le perceptron

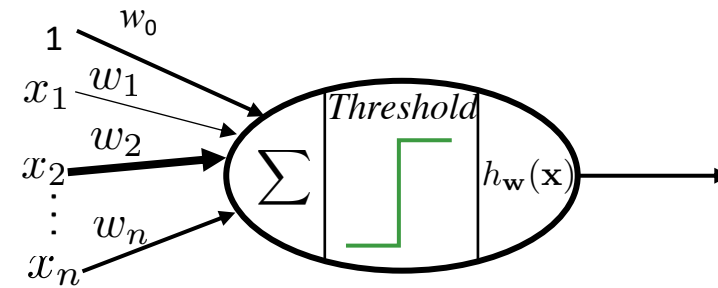
Sample x_i	x_{i1}	x_{i2}	y_i
x_1	4.5	4.7	1
x_2	4.7	4.1	1
x_3	4.7	4.9	1
x_4	5	5.9	1
x_5	5.1	4.9	1
x_6	5.1	5	1
x_7	5.2	3.5	0
..	..	..	..

$$\mathbf{w} = [w_0, w_1, w_2]$$
$$\mathbf{x} = [1, x_1, x_2]$$
$$w \approx [-2.6, 0.2, 1]$$



Algorithme du Perceptron:

- pour chaque paire
 - calculer $h_{\mathbf{w}}(\mathbf{x}_t) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x}_t)$
 - $w_i \leftarrow w_i + \alpha(y_t - h_{\mathbf{w}}(\mathbf{x}_t))x_{t,i} \quad \forall i$
- recommencer à l'étape 1



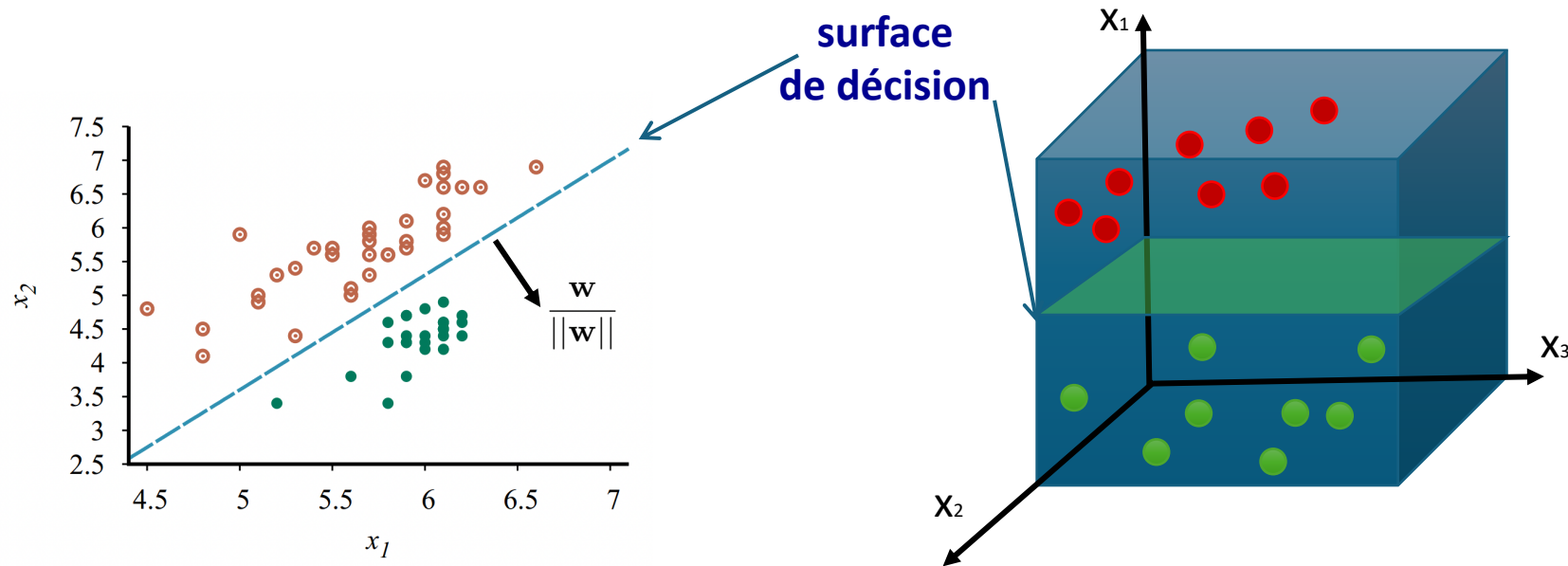
$$\mathbf{w} \cdot \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2$$

Si $\mathbf{w} \cdot \mathbf{x} \geq 0$ alors $y = 1$ sinon $y = 0$

$\text{Threshold}(\mathbf{w} \cdot \mathbf{x})$

Surface de séparation

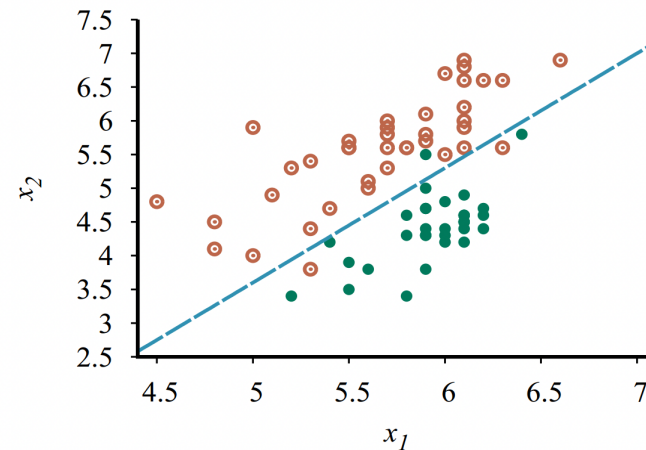
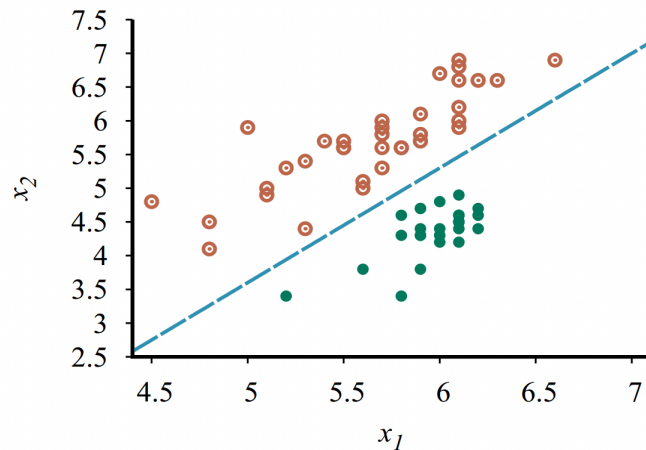
- Le Perceptron cherche donc un **séparateur linéaire** entre les deux classes



- La **surface de décision** d'un classifieur est la surface (dans le cas du perceptron en 2D, une droite) qui sépare les deux régions classifiées dans les deux classes différentes

Convergence et séparabilité

- Si les exemples d'entraînement sont **linéairement séparables** (gauche), l'algorithme est garanti de converger à **une solution avec une erreur nulle** sur l'ensemble d'entraînement, quel que soit le choix de (α)

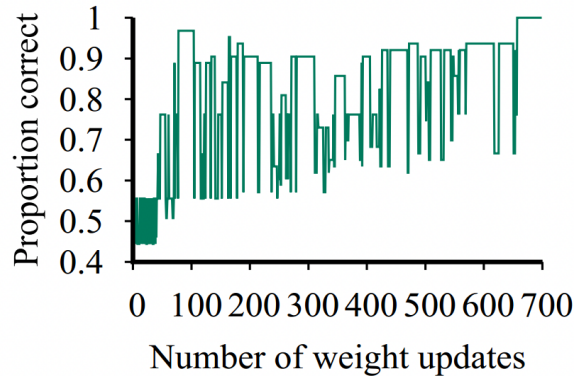


- Si non-séparable linéairement (droite), pour garantir la convergence à une **solution avec la plus petite erreur possible en entraînement**, on doit décroître le taux d'apprentissage, par ex. selon

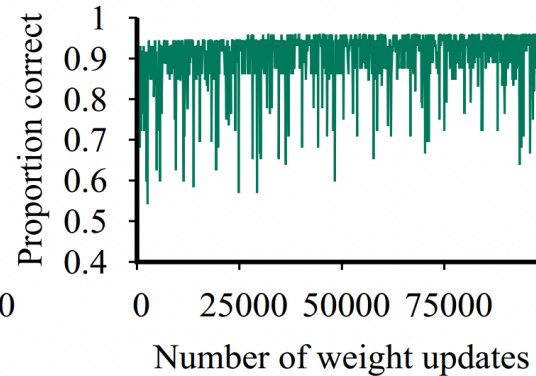
$$\alpha_k = \frac{\alpha}{1 + \beta k}$$

Courbe d'apprentissage

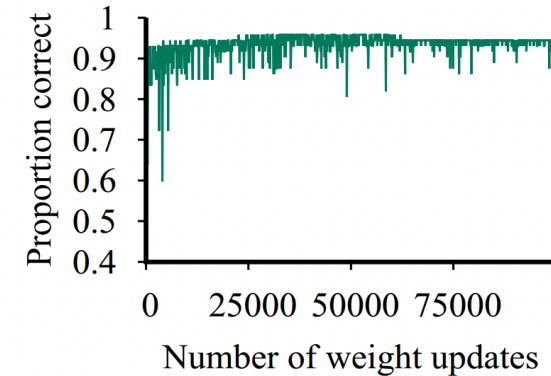
- Pour visualiser la progression de l'apprentissage, on peut regarder la **courbe d'apprentissage**, c'est-à-dire la courbe du taux d'erreur (ou de succès) en fonction du nombre de mises à jour des paramètres



linéairement
séparable, avec taux
d'app. constant



pas linéairement
séparable, avec aux
d'app. constant



pas linéairement
séparable, avec taux
d'app. décroissant

Minimisation de perte

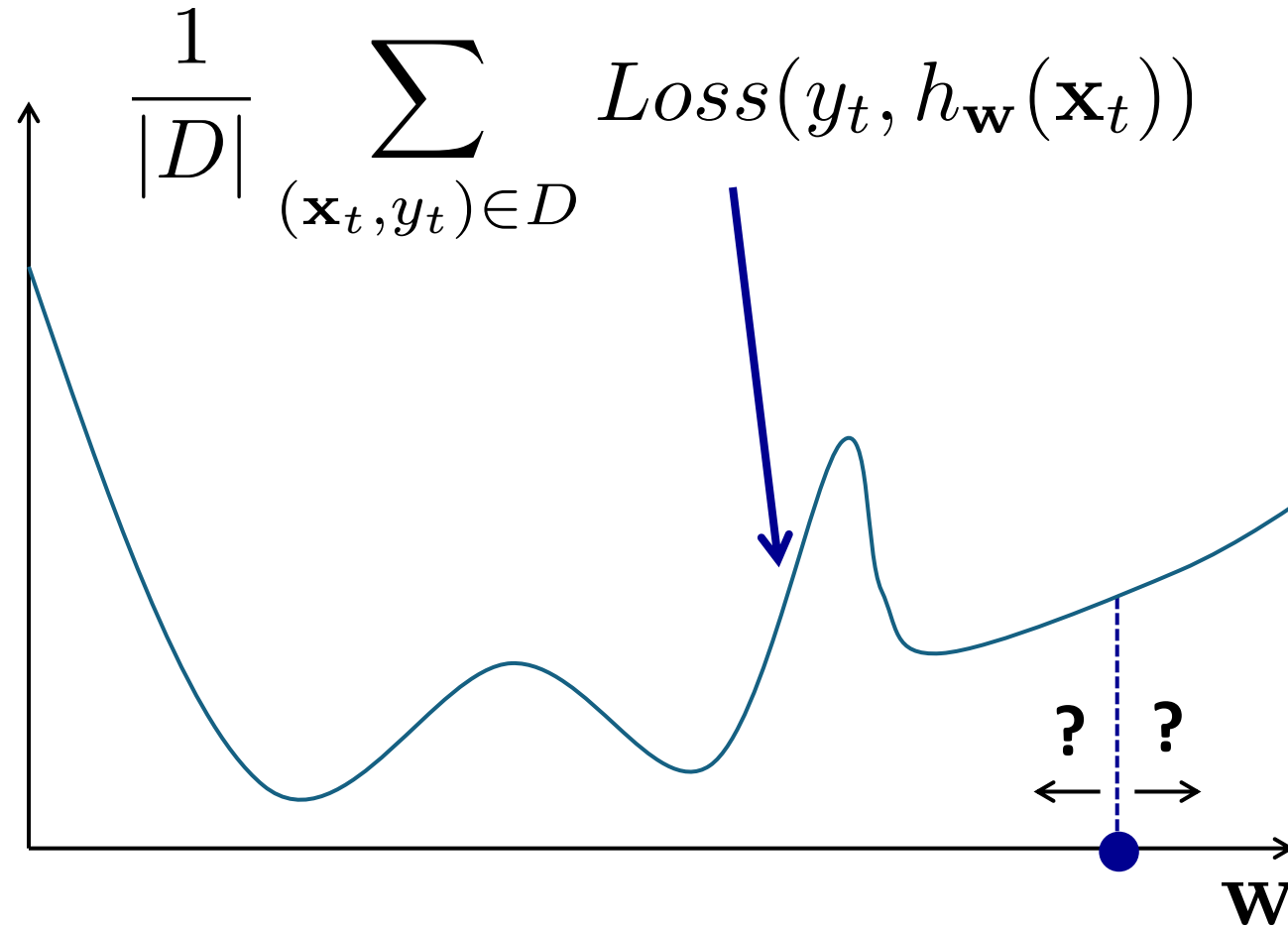
Apprentissage comme la minimisation d'une perte

- Le problème de l'apprentissage peut être formulé comme un problème d'optimisation
 - pour chaque exemple d'entraînement, on souhaite minimiser une certaine distance $[\text{Loss}(\mathbf{y}_t, \mathbf{h}_w(\mathbf{x}_t))]$ entre la cible $[\mathbf{y}_t]$ et la prédiction $\mathbf{h}_w(\mathbf{x}_t)$
 - on appelle cette distance une perte
- Dans le cas du perceptron:

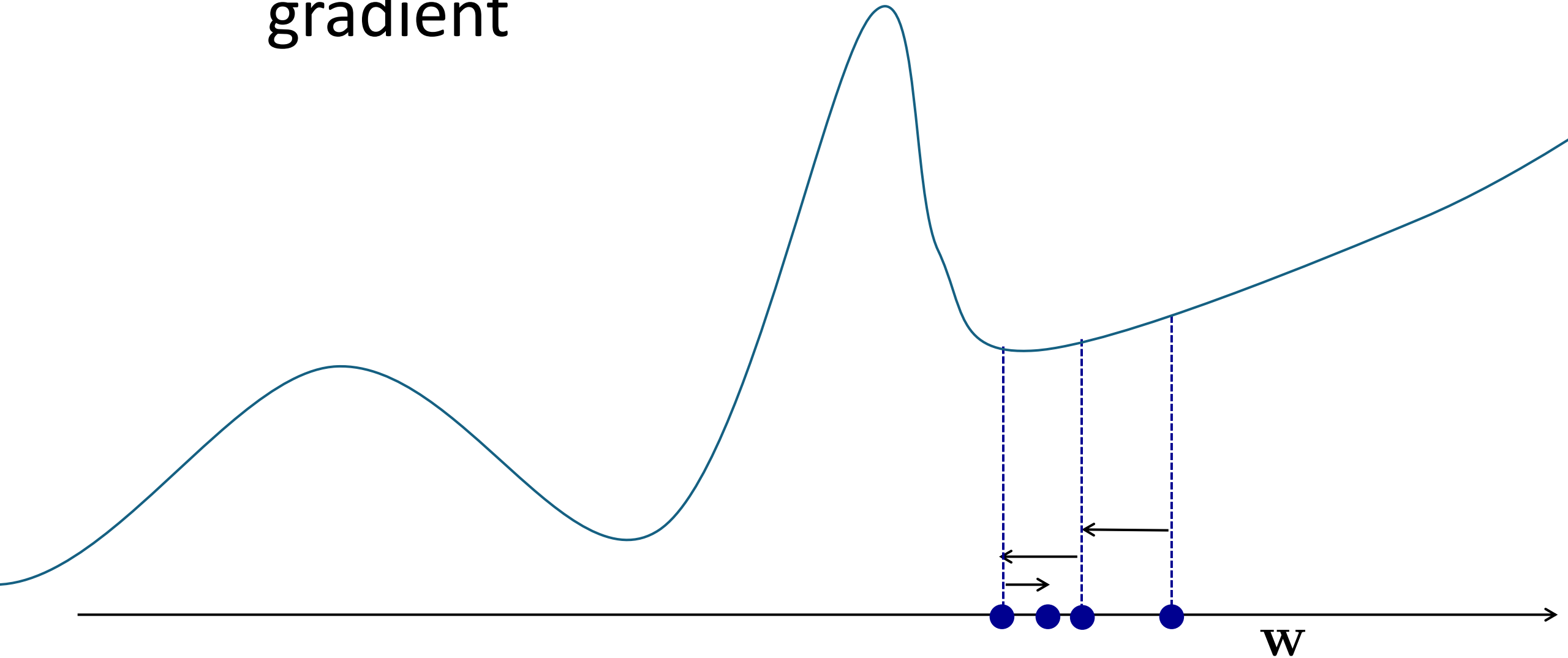
$$\text{Loss}(y_t, h_w(\mathbf{x}_t)) = -(y_t - h_w(\mathbf{x}_t)) \mathbf{w} \cdot \mathbf{x}_t$$

- si la prédiction est bonne, le coût est 0
- si la prédiction est mauvaise, le coût est la distance entre $\{\mathbf{w} \cdot \mathbf{x}_t\}$ et le seuil à franchir pour que la prédiction soit bonne

Recherche locale pour la minimisation d'une perte

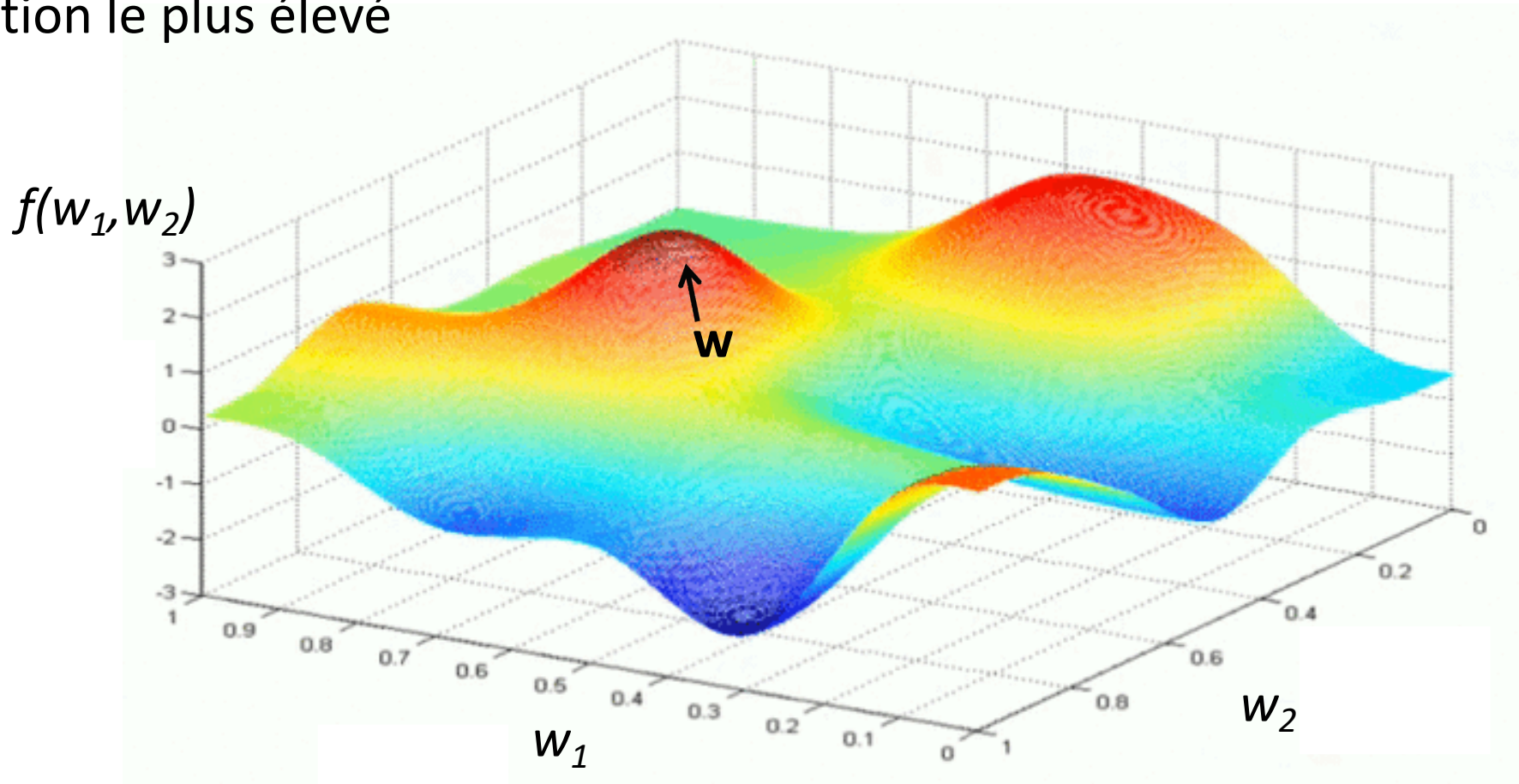


Algorithme de descente de gradient



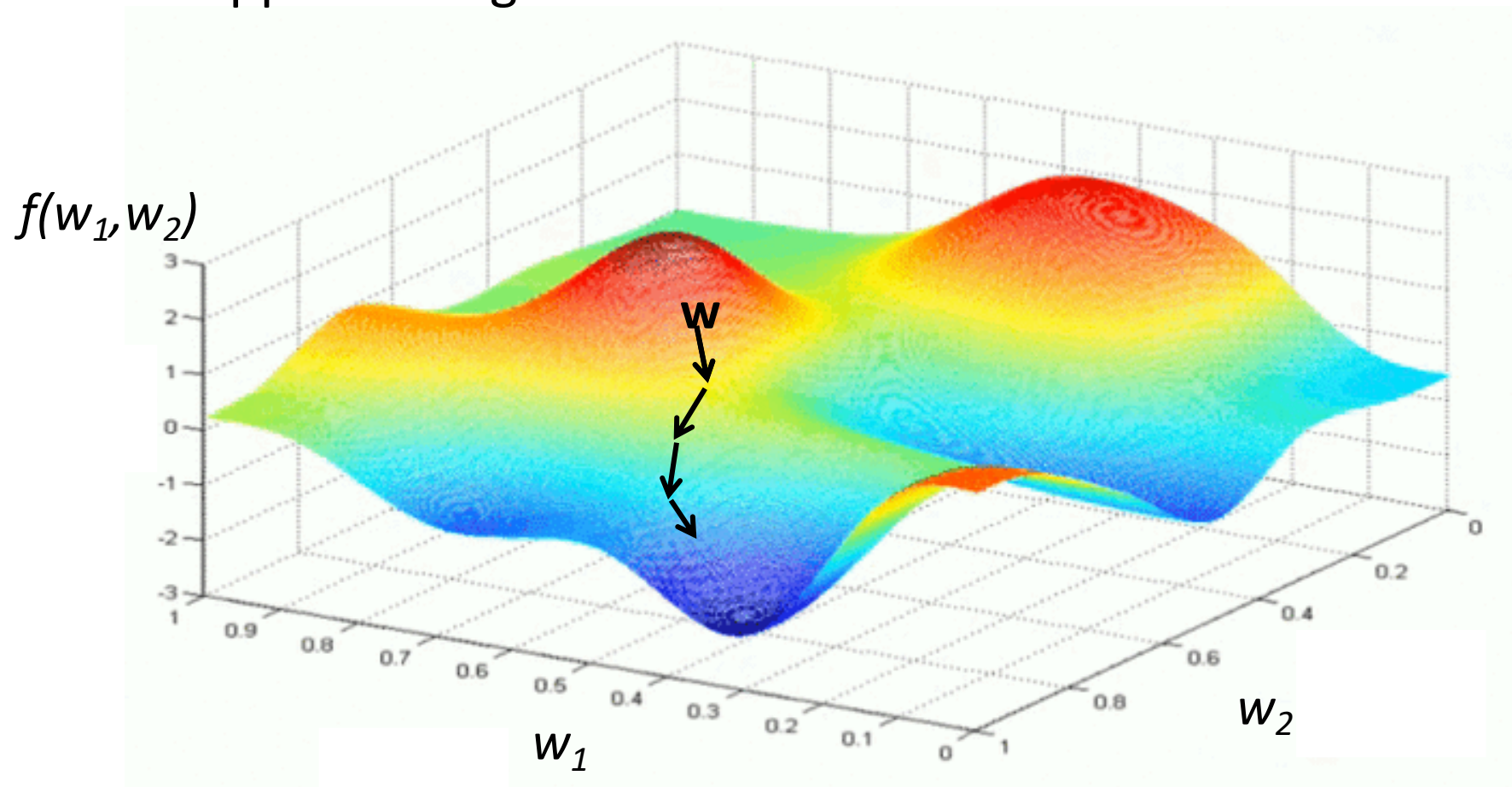
Descente de gradient

- Le gradient donne la direction (vecteur) ayant le taux d'accroissement de la fonction le plus élevé



Descente de gradient

- La direction opposée au gradient nous donne la direction à suivre



Apprentissage vue comme la minimisation d'une perte

- En apprentissage automatique, on souhaite optimiser:

$$\frac{1}{|D|} \sum_{(\mathbf{x}_t, y_t) \in D} Loss(y_t, h_{\mathbf{w}}(\mathbf{x}_t))$$

- Le gradient par rapport à la perte moyenne contient les dérivées partielles:

$$\frac{1}{|D|} \sum_{(\mathbf{x}_t, y_t) \in D} \frac{\partial}{\partial w_i} Loss(y_t, h_{\mathbf{w}}(\mathbf{x}_t)) \quad \forall i$$

- Devrait calculer la moyenne des dérivées sur tous les exemples d'entraînement avant de faire une mise à jour des paramètres!

Descente de gradient

- **La descente de gradient** est un algorithme pour minimiser la perte lors de l'entraînement d'un réseau de neurone.
- La descente de gradient normale consiste à mettre à jour les paramètres en tenant compte du gradient moyen (c.-à-d. des dérivées partielles) de **tous les exemples** d'entraînement dans l'ensemble D:

- Initialiser **W** aléatoirement
- Répéter jusqu'à la convergence

$$\bullet \quad w_i \leftarrow w_i - \alpha * \frac{1}{|D|} \sum_{(\mathbf{x}_t, y_t) \in D} \frac{\partial}{\partial w_i} \text{Loss}(y_t, h_{\mathbf{w}}(\mathbf{x}_t)) \quad \forall i$$

Descente de gradient stochastique

- La **descente de gradient stochastique** sélectionne un exemple d'entraînement au hasard, calcule le gradient par rapport à ce **seul exemple** et met à jours tous les poids.

- Initialiser $\mathbf{W}$ aléatoirement
- Répéter jusqu'à la convergence
 - Choisir un exemple d'entraînement $(\mathbf{x}_t, y_t)$ **au hasard**

$$- w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(y_t, h_{\mathbf{w}}(\mathbf{x}_t)) \quad \forall i$$

- Cette procédure est beaucoup plus efficace lorsque l'ensemble d'entraînement $\mathbf{D}$ est grand

Descente de gradient par mini-batch

- La **descente de gradient par mini-batch** calcule le gradient par rapport à un **petit sous ensemble de données** et met à jours tous les poids.

- Initialiser **W** aléatoirement
- Répéter jusqu'à la convergence
 - Choisir un sous-ensemble de données (mini-batch)
 - Calculer le gradient basé sur le mini-batch
 - Mettre à jour tous les poids en fonction du gradient

- Cette procédure est un compromis entre la descente du gradient standard et la descente du gradient stochastique

Retour sur le Perceptron

Rappel

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(x)}{\partial g(x)} \frac{\partial g(x)}{\partial x} \quad \frac{\partial x^2}{\partial x} = 2x \quad \frac{\partial x^2}{\partial x} = 1$$

- Utilisons le gradient (dérivée partielle) pour déterminer une direction de mise à jour des paramètres:

$$\begin{aligned} \frac{\partial}{\partial w_i} \text{Loss}(y_t, h_{\mathbf{w}}(\mathbf{x}_t)) &= \frac{\partial}{\partial w_i} (y_t - h_{\mathbf{w}}(x_t))^2 = 2(y_t - h_{\mathbf{w}}(x_t)) \times \frac{\partial}{\partial w_i} (y_t - h_{\mathbf{w}}(x_t)) \\ &= -2(y_t - h_{\mathbf{w}}(x_t)) \times x_{t,i} \end{aligned} \quad \begin{matrix} \searrow \\ \sum_j \omega_j x_{t,j} \end{matrix}$$

- Rappel de la règle de mise à jour de la descente du gradient

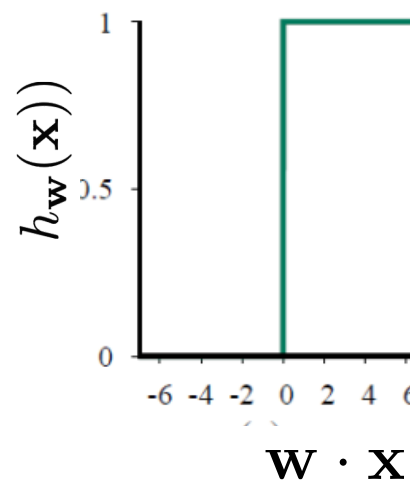
$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(y_t, h_{\mathbf{w}}(\mathbf{x}_t)) \quad \forall i$$

- On obtient à nouveau la règle d'apprentissage du Perceptron

$$w_i \leftarrow w_i + \alpha (y_t - h_{\mathbf{w}}(\mathbf{x}_t)) x_{t,i} \quad \forall i$$

Apprentissage vue comme la minimisation d'une perte

- La procédure de descente de gradient stochastique est applicable à n'importe quelle perte dérivable partout
- Dans le cas du Perceptron, on a un peu triché:
 - la dérivée de $\mathbf{h}_{\mathbf{w}}(\mathbf{x})$ n'est pas définie lorsque $\mathbf{w} \cdot \mathbf{x} = 0$
- L'utilisation de la fonction *Threshold* (qui est constante par partie) fait que la courbe d'entraînement peut être instable



Rappel sur les dérivées partielles et le gradient

Dérivées

- On peut obtenir la direction de descente via la **dérivée**

$$f'(a) = \frac{df(a)}{da} = \lim_{\Delta \rightarrow 0} \frac{f(a + \Delta) - f(a)}{\Delta}$$

- Le signe de la dérivée est la **direction d'augmentation** de
 - signe positif indique que **$f(a)$** augmente lorsque (**a**) augmente
 - signe négatif indique que **$f(a)$** diminue lorsque (**a**) augmente
- La valeur absolue de la dérivée est le **taux d'augmentation** de **f**
- Plutôt que **d** , je vais utiliser le symbole **∂**

Dérivées

- Les dérivées usuelles les plus importantes sont les suivantes:

$$\frac{\partial a}{\partial x} = 0$$

$$\frac{\partial x^n}{\partial x} = nx^{n-1}$$

$$\frac{\partial \log(x)}{\partial x} = \frac{1}{x}$$

$$\frac{\partial \exp(x)}{\partial x} = \exp(x)$$

a et ***n*** sont
des constantes

Dérivées

- On peut obtenir des dérivées de composition de fonctions

$$\frac{\partial a f(x)}{\partial x} = a \frac{\partial f(x)}{\partial x}$$

$$\frac{\partial f(x)^n}{\partial x} = n f(x)^{n-1} \frac{\partial f(x)}{\partial x}$$

$$\frac{\partial \exp(f(x))}{\partial x} = \exp(f(x)) \frac{\partial f(x)}{\partial x}$$

$$\frac{\partial \log(f(x))}{\partial x} = \frac{1}{f(x)} \frac{\partial f(x)}{\partial x}$$

a et ***n*** sont
des constantes

Dérivées

- Exemple 1: $f(x) = 3x^4$

$$\frac{\partial f(x)}{\partial x} = \frac{\partial 3x^4}{\partial x} = 3 \frac{\partial x^4}{\partial x} = 12x^3$$

Dérivées

- Exemple 2: $f(x) = \exp\left(\frac{x^2}{3}\right)$

$$\frac{\partial f(x)}{\partial x} = \frac{\partial \exp\left(\frac{x^2}{3}\right)}{\partial x} = \exp\left(\frac{x^2}{3}\right) \frac{\partial \frac{x^2}{3}}{\partial x}$$

$$= \frac{1}{3} \exp\left(\frac{x^2}{3}\right) \frac{\partial x^2}{\partial x} = \frac{2}{3} \exp\left(\frac{x^2}{3}\right) x$$

Dérivées

- Pour des combinaisons plus complexes:

$$\frac{\partial g(x) + h(x)}{\partial x} = \frac{\partial g(x)}{\partial x} + \frac{\partial h(x)}{\partial x}$$

$$\frac{\partial g(x)h(x)}{\partial x} = \frac{\partial g(x)}{\partial x} h(x) + g(x) \frac{\partial h(x)}{\partial x}$$

$$\frac{\partial \frac{g(x)}{h(x)}}{\partial x} = \frac{\partial g(x)}{\partial x} \frac{1}{h(x)} - \frac{g(x)}{h(x)^2} \frac{\partial h(x)}{\partial x}$$

Dérivées

- Exemple 3: $f(x) = x \exp(x)$

$$\begin{aligned}\frac{\partial f(x)}{\partial x} &= \frac{\partial x}{\partial x} \exp(x) + x \frac{\partial \exp(x)}{\partial x} \\ &= \exp(x) + x \exp(x)\end{aligned}$$

Dérivées

- Exemple 4: $f(x) = \frac{\exp(x)}{x}$

$$\begin{aligned}\frac{\partial f(x)}{\partial x} &= \frac{\partial \exp(x)}{\partial x} \frac{1}{x} - \frac{\exp(x)}{x^2} \frac{\partial x}{\partial x} \\ &= \frac{\exp(x)}{x} - \frac{\exp(x)}{x^2}\end{aligned}$$

Dérivées

- Exemple 4: $f(x) = \frac{\exp(x)}{x}$

dérivation alternative!

$$\begin{aligned}\frac{\partial f(x)}{\partial x} &= \frac{\partial \exp(x)}{\partial x} \frac{1}{x} + \exp(x) \frac{\partial \frac{1}{x}}{\partial x} \\ &= \frac{\exp(x)}{x} - \frac{\exp(x)}{x^2}\end{aligned}$$

Dérivée partielle et gradient

- Dans notre cas, la fonction à optimiser dépend de plus d'une variable
 - elle dépend de tout le vecteur $\mathbf{w}$
- Dans ce cas, on va considérer les **dérivées partielles**, c.-à-d. la dérivée par rapport à chacune des variables en supposant que les autres sont constantes:

$$\frac{\partial f(a, b)}{\partial a} = \lim_{\Delta \rightarrow 0} \frac{f(a + \Delta, b) - f(a, b)}{\Delta}$$

$$\frac{\partial f(a, b)}{\partial b} = \lim_{\Delta \rightarrow 0} \frac{f(a, b + \Delta) - f(a, b)}{\Delta}$$

Dérivée partielle et gradient

- Exemple de fonction à deux variables:

$$f(x, y) = \frac{x^2}{y}$$

- Dérivées partielles:

$$\frac{\partial f(x, y)}{\partial x} = \frac{2x}{y}$$

traite y
comme une
constante

$$\frac{\partial f(x, y)}{\partial y} = \frac{-x^2}{y^2}$$

traite x
comme une
constante

Dérivée partielle et gradient

- Un deuxième exemple:

$$f(\mathbf{x}) = \frac{\exp(x_2)}{\exp(x_1) + \exp(x_2) + \exp(x_3)}$$

- Dérivée partielle $\frac{\partial f(\mathbf{x})}{\partial x_1}$:

équivalent à faire la dérivée de $f(x) = \frac{a}{\exp(x) + b}$

où $x = x_1$

- et on a des **constantes** $a = \exp(x_2)$ et $b = \exp(x_2) + \exp(x_3)$

Dérivée partielle et gradient

- Un deuxième exemple: $f(x) = \frac{a}{\exp(x) + b}$

$$\frac{\partial f(x)}{\partial x} = \frac{\partial}{\partial x} \frac{a}{\exp(x) + b} = a \frac{\partial}{\partial x} \frac{1}{\exp(x) + b}$$

$$= \frac{-a}{(\exp(x) + b)^2} \frac{\partial}{\partial x} (\exp(x) + b)$$

$$= \frac{-a \exp(x)}{(\exp(x) + b)^2}$$

Dérivée partielle et gradient

- Un deuxième exemple:

$$\frac{\partial f(x)}{\partial x} = \frac{-a \exp(x)}{(\exp(x) + b)^2}$$

où $x = x_1$, $a = \exp(x_2)$ $b, = \exp(x_2) + \exp(x_3)$

- On remplace:
$$\frac{\partial f(\mathbf{x})}{\partial x_1} = \frac{-\exp(x_2) \exp(x_1)}{(\exp(x_1) + \exp(x_2) + \exp(x_3))^2}$$

Dérivée partielle et gradient

- Un troisième exemple:

$$f(\mathbf{x}) = \frac{\exp(x_2)}{\exp(x_1) + \exp(x_2) + \exp(x_3)}$$

- Dérivée partielle $\frac{\partial f(\mathbf{x})}{\partial x_2}$:

équivalent à faire la dérivée de $f(x) = \frac{\exp(x)}{\exp(x) + b}$

où $x = x_2$

et on a une constante $b = \exp(x_1) + \exp(x_3)$

Dérivée partielle et gradient

- Un troisième exemple: $f(x) = \frac{\exp(x)}{\exp(x) + b}$

$$\begin{aligned}\frac{\partial f(x)}{\partial x} &= \frac{\partial \exp(x)}{\partial x} \frac{1}{\exp(x) + b} - \frac{\exp(x)}{(\exp(x) + b)^2} \frac{\partial(\exp(x) + b)}{\partial x} \\ &= \frac{\exp(x)}{\exp(x) + b} - \frac{\exp(x) \exp(x)}{(\exp(x) + b)^2}\end{aligned}$$

Dérivée partielle et gradient

- Un troisième exemple :

$$\frac{\partial f(x)}{\partial x} = \frac{\exp(x)}{\exp(x) + b} - \frac{\exp(x) \exp(x)}{(\exp(x) + b)^2}$$

où $x = x_2$, $b = \exp(x_1) + \exp(x_3)$

- On remplace:

$$\frac{\partial f(\mathbf{x})}{\partial x_2} = \frac{\exp(x_2)}{\exp(x_2) + \exp(x_1) + \exp(x_3)} - \frac{\exp(x_2) \exp(x_2)}{(\exp(x_2) + \exp(x_1) + \exp(x_3))^2}$$

Dérivée partielle et gradient

- On va appeler **gradient** ∇f d'une fonction f le vecteur contenant les dérivées partielles de f par rapport à toutes les variables
- Dans l'exemple avec la fonction $f(x, y)$:

$$\begin{aligned}\nabla f(x, y) &= \left[\frac{\partial f(x, y)}{\partial x}, \frac{\partial f(x, y)}{\partial y} \right] \\ &= \left[\frac{2x}{y}, \frac{-x^2}{y^2} \right]\end{aligned}$$